

Data Vortex 2026 - Phase 2 Selected SQL Queries

Selected questions

Difficulty	Question
Easy	E3 - Average Engagement by Platform
Medium	M2 - Do High Follower Users Get More Engagement?
Hard	H3 - Platform Performance Compared With Its Own Average

The following queries are executed dynamically against SQLite tables created from the cleaned users and posts datasets.

E3 - Average Engagement by Platform

Challenge

Calculate the average likes, shares, and comments for each platform. Determine which platform generates the highest average total engagement.

SQL query

```
WITH platform_engagement AS (  
  SELECT  
    platform,  
  
    COUNT(*) AS post_count,  
  
    AVG(likes) AS average_likes,  
    AVG(shares) AS average_shares,  
    AVG(comments) AS average_comments,  
  
    AVG(  
      COALESCE(likes, 0) + shares + comments  
    ) AS average_total_engagement  
  
  FROM posts  
  
  WHERE platform IS NOT NULL  
  
  GROUP BY platform  
)  
  
SELECT  
  platform,  
  post_count,
```

```
ROUND(average_likes, 2) AS average_likes,  
ROUND(average_shares, 2) AS average_shares,  
ROUND(average_comments, 2) AS average_comments,  
ROUND(average_total_engagement, 2)  
    AS average_total_engagement
```

```
FROM platform_engagement
```

```
ORDER BY average_total_engagement DESC;
```

Query result

The result is ordered from the highest to the lowest average total engagement. Reddit produced the highest average total engagement in this dataset.

M2 - Do High Follower Users Get More Engagement?

Challenge

Divide users into two groups:

- High follower users: at least 25,000 followers.
- Low follower users: fewer than 25,000 followers.

Compare their average engagement per post.

SQL query

```
WITH post_engagement AS (  
    SELECT  
        post_id,  
        user_id,  
  
        COALESCE(likes, 0) + shares + comments  
        AS total_engagement  
  
    FROM posts  
)  
  
user_groups AS (  
    SELECT  
        user_id,  
  
        CASE  
            WHEN follower_count >= 25000  
            THEN 'High follower (>=25000)'  
            ELSE 'Low follower (<25000)'  
        END AS follower_group
```

```
FROM users
),
group_summary AS (
    SELECT
        user_groups.follower_group,

        COUNT(post_engagement.post_id)
        AS post_count,

        COUNT(DISTINCT user_groups.user_id)
        AS user_count,

        AVG(post_engagement.total_engagement)
        AS average_engagement_per_post

    FROM user_groups

    LEFT JOIN post_engagement
        ON user_groups.user_id = post_engagement.user_id

    GROUP BY user_groups.follower_group
)

SELECT
    follower_group,
    post_count,
    user_count,
    ROUND(average_engagement_per_post, 2)
    AS average_engagement_per_post

FROM group_summary

ORDER BY average_engagement_per_post DESC;
```

Query result

The low-follower group produced the higher average engagement per post:

- Low-follower users: 3,535.84
- High-follower users: 3,508.45

H3 - Platform Performance Compared With Its Own Average

Challenge

For every platform, identify posts whose engagement is at least twice the average engagement of that platform.

SQL query

```
WITH post_engagement AS (  
  SELECT  
    post_id,  
    user_id,  
    platform,  
    timestamp,  
  
    COALESCE(likes, 0) + shares + comments  
      AS total_engagement  
  
  FROM posts  
  
  WHERE platform IS NOT NULL  
) ,  
  
platform_averages AS (  
  SELECT  
    platform,  
  
    AVG(total_engagement)  
      AS platform_average_engagement  
  
  FROM post_engagement  
  
  GROUP BY platform  
)  
  
SELECT  
  post_engagement.platform,  
  post_engagement.post_id,  
  post_engagement.user_id,  
  post_engagement.timestamp,  
  post_engagement.total_engagement,  
  
  ROUND(  
    platform_averages.platform_average_engagement,  
    2  
  ) AS platform_average_engagement,  
  
  ROUND(  
    post_engagement.total_engagement  
    / platform_averages.platform_average_engagement,  
    2  
  ) AS multiple_of_platform_average  
  
FROM post_engagement  
  
INNER JOIN platform_averages  
  ON post_engagement.platform = platform_averages.platform
```

```
WHERE post_engagement.total_engagement
      >= 2 * platform_averages.platform_average_engagement

ORDER BY
  post_engagement.platform,
  post_engagement.total_engagement DESC;
```

Query result

The query identified 139 posts whose total engagement was at least twice the average engagement of their own platform.

Data-handling assumption

Missing likes are treated as zero only when calculating total engagement. This keeps posts in the analysis. The original cleaned dataset is not modified.

Posts with missing platform values are excluded from platform-specific comparisons because they cannot be assigned to a named platform.